4. Worker 노드 구현

4.1 Worker 노드 역할

Worker 노드는 Narwhal 시스템에서 트랜잭션 처리와 배치 생성을 담당하는 구성 요소입니다.

4.1.1 주요 책임

- **트랜잭션 수집**: 클라이언트로부터 트랜잭션 수신
- **배치 생성**: 트랜잭션들을 배치로 묶어서 처리
- **배치 전파**: 다른 Worker들에게 배치 전송
- **가용성 보장**: Primary가 배치를 참조할 수 있도록 저장

4.1.2 아키텍처 특징

- **Primary와 분리**: 트랜잭션 처리와 컨센서스 로직 분리
- **병렬 처리**: 여러 Worker가 독립적으로 동작
- **확장성**: Worker 수를 늘려 처리량 증대 가능

4.2 배치 생성 메커니즘

4.2.1 배치 구조

```
pub struct Batch {
pub transactions: Vec<Transaction>,
```

```rust
pub author: WorkerId,
pub timestamp: SystemTime,
}

pub struct BatchDigest(pub [u8; 32]);

// 배치와 다이제스트의 매핑
pub type BatchStore = HashMap<BatchDigest, Batch>;
```

4.2.2 배치 생성 정책

- **크기 기반**: 설정된 최대 크기에 도달하면 배치 생성
- **시간 기반**: 일정 시간 경과 후 강제 배치 생성
- **트랜잭션 수 기반**: 특정 개수의 트랜잭션 누적 시 배치 생성

```rust
pub struct BatchMaker {
max_batch_size: usize,
max_batch_delay: Duration,
pending_transactions: Vec<Transaction>,
last_batch_time: Instant,
}

impl BatchMaker {
pub fn should_make_batch(&self) -> bool {
```

```
self.pending_transactions.len() * TRANSACTION_SIZE >= self.max_batch_size
|| self.last_batch_time.elapsed() >= self.max_batch_delay
}
}
```

4.3 트랜잭션 처리 파이프라인

4.3.1 수신 단계

```
pub struct TransactionReceiver {
rx_client: Receiver<Transaction>,
pending_queue: VecDeque<Transaction>,
metrics: WorkerMetrics,
}

impl TransactionReceiver {
pub async fn receive_transactions(&mut self) {
while let Some(transaction) = self.rx_client.recv().await {
// 트랜잭션 유효성 검사
if self.validate_transaction(&transaction) {
self.pending_queue.push_back(transaction);
self.metrics.transactions_received.inc();
}
```

```
}
}
}
```

4.3.2 검증 단계

- **형식 검증**: 트랜잭션 구조의 유효성 확인
- **서명 검증**: 트랜잭션 서명의 암호학적 검증
- **중복 검사**: 이미 처리된 트랜잭션인지 확인
- **스팸 방지**: 동일 발신자의 과도한 트랜잭션 제한

4.4 배치 전파 시스템

4.4.1 전파 메커니즘

```
pub struct BatchBroadcaster {
network: WorkerNetwork,
committee: Committee,
worker_id: WorkerId,
}

impl BatchBroadcaster {
pub async fn broadcast_batch(&self, batch: Batch) -> Result<(), NetworkError>
let digest = batch.digest();
let message = WorkerMessage::Batch(batch);
```

```
// 모든 다른 Worker들에게 전송
for worker_info in self.committee.workers() {
if worker_info.id != self.worker_id {
self.network.send(worker_info.address, message.clone()).await?;
}
}
Ok(())
}
}
```

4.4.2 전파 최적화

- **Gossip Protocol**: 효율적인 배치 전파
- **Redundancy Control**: 중복 전송 방지
- **Priority Queuing**: 중요한 배치 우선 전송

4.5 저장소 관리

4.5.1 배치 저장소

```
pub struct WorkerStorage {
batch_store: HashMap<BatchDigest, Batch>,
transaction_index: HashMap<TransactionDigest, BatchDigest>,
gc_policy: GarbageCollectionPolicy,
```

```
}

impl WorkerStorage {
pub fn store_batch(&mut self, batch: Batch) -> BatchDigest {
let digest = batch.digest();
// 배치 저장
self.batch_store.insert(digest, batch.clone());
// 트랜잭션 인덱스 업데이트
for tx in &batch.transactions {
self.transaction_index.insert(tx.digest(), digest);
}
digest
}
pub fn get_batch(&self, digest: &BatchDigest) -> Option<&Batch> {
self.batch_store.get(digest)
}
}
```

4.5.2 가비지 컬렉션

- **참조 카운팅**: Primary의 참조 여부 추적
- **시간 기반 정리**: 오래된 배치 자동 삭제
- **용량 기반 정리**: 저장소 용량 초과 시 정리

4.6 Primary와의 상호작용

4.6.1 배치 요청 처리

```
pub struct BatchRequestHandler {
storage: WorkerStorage,
network: WorkerNetwork,
}

impl BatchRequestHandler {
pub async fn handle_batch_request(&self,
request: BatchRequest
) -> Result<(), HandlerError> {
match self.storage.get_batch(&request.digest) {
Some(batch) => {
let response = BatchResponse {
digest: request.digest,
batch: batch.clone(),
};
self.network.send_to_primary(response).await?;
}
None => {
// 배치를 찾을 수 없음을 알림
```

```
let error = BatchNotFound { digest: request.digest };
self.network.send_to_primary(error).await?;
}
}
Ok(())
}
}
```

4.6.2 동기화 지원

- **배치 제공**: Primary의 요청에 따른 배치 전송
- **가용성 보고**: 보유한 배치 목록 주기적 보고
- **복구 지원**: Primary 복구 시 배치 재전송

4.7 부하 분산과 확장성

4.7.1 Worker 분산

```
pub struct WorkerPool {
workers: Vec<Worker>,
load_balancer: LoadBalancer,
coordinator: WorkerCoordinator,
}

impl WorkerPool {
```

```
pub fn assign_transaction(&mut self, transaction: Transaction) -> WorkerId {
// 부하 분산 알고리즘에 따른 Worker 선택
self.load_balancer.select_worker(&transaction)
}
}
```

4.7.2 확장 전략

- **수평 확장**: Worker 수 증가로 처리량 향상
- **샤딩**: 트랜잭션 타입별 Worker 분할
- **동적 조정**: 부하에 따른 Worker 수 자동 조정

4.8 성능 최적화

4.8.1 배치 최적화

```
pub struct BatchOptimizer {
target_batch_size: usize,
compression_enabled: bool,
deduplication_enabled: bool,
}

impl BatchOptimizer {
pub fn optimize_batch(&self, transactions: Vec<Transaction>) -> Batch {
```

```
let mut optimized_txs = transactions;
if self.deduplication_enabled {
optimized_txs = self.remove_duplicates(optimized_txs);
}
if self.compression_enabled {
optimized_txs = self.compress_transactions(optimized_txs);
}
Batch::new(optimized_txs)
}
}
```

4.8.2 메모리 관리

- **스트리밍 처리**: 대용량 배치의 스트림 처리
- **압축**: 배치 데이터 압축으로 메모리 절약
- **풀링**: 객체 재사용으로 할당 비용 절감

4.9 장애 복구

4.9.1 배치 복구

```
pub struct BatchRecovery {
storage: WorkerStorage,
network: WorkerNetwork,
recovery_timeout: Duration,
```

```
}

impl BatchRecovery {
pub  async  fn  recover_missing_batches(&mut  self,
missing_digests: Vec<BatchDigest>
) -> Result<(), RecoveryError> {
for  digest  in  missing_digests {
// 다른 Worker들로부터 배치 요청
if  let Some(batch) = self.request_batch_from_peers(digest).await? {
self.storage.store_batch(batch);
}
}
Ok(())
}
}
```

4.9.2 복구 전략

- **피어 복구**: 다른 Worker로부터 배치 복구
- **체크포인트**: 주기적 상태 저장
- **점진적 복구**: 중요한 배치부터 우선 복구

Worker 노드는 이러한 메커니즘들을 통해 높은 처리량과 안정성을 제공하며, Primary 노드와 함께 Narwhal 시스템의 성능을 극대화합니다.